



Fault-Tolerant Distributed Query System

Team: RRR

Rohith Reddy	NU 002641044
Ruby Wang	NU 002610125
Ruchika Shashidhara	NU 002245068

Project Description



Develop a distributed operation that partitions a large dataset and distributes these partitions—with replicas—across worker actors.

Dataset: GitHub Events API Payloads (GH Archive)

Processing: Supports broadcasting of small datasets as needed.

Execution: On query submission, the system routes the query to the relevant partitioned actors, processes locally, and aggregates the results.

Project Architecture



- Coordinator Actor: Receives and splits queries into subqueries, Aggregates results from worker actors
- Partition Manager Actor: Partitions the large dataset using defined criteria. Manages assignment of primary and replica partitions
- Worker Actors: Execute subqueries on local partitioned data. Maintain replicas for fault tolerance
- Distributed Query Processing: Intelligent routing and aggregation of partial results
- Resilience & Monitoring: Automatic failover. Alerts for processing delays or failures

Use Cases

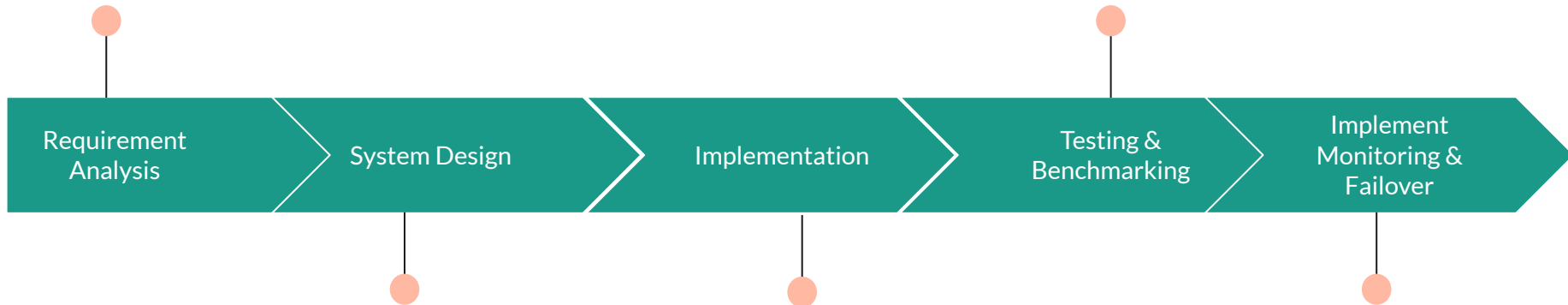
- Real-Time Analytics: Analyze GitHub events (pushes, pull requests, issues) to generate actionable insights
- Scalable Data Processing: Execute distributed queries over large datasets efficiently
- Fault-Tolerant Operations: Ensure high availability with automated failover and data replication

Methodology

Define data characteristics, performance metrics, and SLA targets.

Unit and integration tests using Akka TestKit/Cluster TestKit.

Implement performance benchmarking using Gatling



Create architecture diagrams and design the actor-based model.

Develop Partition Manager, Coordinator, and Worker actors using Akka.

Incorporate replication, distributed query routing, and aggregation.

Implement alerts and automatic recovery mechanisms

Data sources

- Dataset: GitHub Events API Payloads - <https://www.gharchive.org/>
- Standardized GitHub event objects with common fields - [GitHub Events](#)
- Includes metadata like event ID, type, actor, repository, payload, and timestamp.
- Captures a variety of GitHub actions (e.g., pushes, pull requests, issues).
- Type: Uncompressed JSON
- Size: 650 MB
- Total Number of Rows: 197139

Sprint Planning



Week	Ruchika	Ruby	Rohith
1 (3/24)	Requirements analysis & project setup	Architecture design & data model	Research on Akka & initial prototyping
2 (4/1)	Develop Partition Manager Actor	Implement Coordinator Actor	Create Worker Actor & define message protocols
3 (4/8)	Integrate dataset partitioning logic	Develop query routing mechanism	Implement replication & failover logic
4 (4/15)	Write unit/integration tests	Benchmark performance & optimize	Implement monitoring & alerting system
5 (4/22)	Final integration & code review	Documentation & sprint wrap-up	Final testing, deployment simulation

Programming Aspects

- All code will be written in Scala
- Project Repository -

<https://github.com/rohithreddykota/akka-query-system>

Acceptance Criteria



- **Average Processing Time:** time < 50 ms; 90th percentile < 100 ms per actor
- Should be able to process ≥ 500 messages/second per actor
- **Average Mailbox size:** < 20 messages per actor for 95% of the time
- References: <https://doc.akka.io/libraries/akka-insights/current/instrumentations/akka/akka.html>

Goals

- Have a strong understanding of and practical experience with Scala and Akka
- Efficiently partition & replicate 650MB data
- Meet SLA for distributed processing.
- Deliver a robust, fault-tolerant distributed system that processes GitHub events



Thank you